

STACK QNO:9

QUESTION:

Question description

Rathik organized technical round interview in Macrosoft for the set of computer science candidates.

The problem is to perform Implement a stack using single queue. you have to use queue data structure, the task is to implement stack using only given queue data structure.

Rathik have given the deadline of only 5 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit ?

Function Description

x is the element to be pushed and s is stack

push(s, x)

- 1) Let size of q be s.
- 2) Enqueue x to q
- 3) One by one Dequeue s items from queue and enqueue them.

Removes an item from stack

pop(s)

1) Dequeue an item from q

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into stack and m is the number of pop operation need to be performed

next line indicates the n number stack elements

Output Format:

First line indicates top of the element of the stack

second line indicates the top of the element after the pop operation

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
1 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 3
```

Test Case 2

INPUT (STDIN)

```
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 11
```

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<code>void Stack::push(int val)</code>

Test Case 2
KEYWORD
<code>q.push(val)</code>

Test Case 3
KEYWORD
<code>void Stack::pop()</code>

Test Case 4
KEYWORD
<code>q.pop();</code>

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
2

Test Case 2
TOKEN COUNT
320

Test Case 3
NLOC
60

Source Code:

```
#include <stdio.h>
int q[10005];
int front = 0;
int rear = 0;

void stack_push(int val) {
    int s = rear - front;
    q[rear++] = val;
    for (int i = 0; i < s; i++) {
        int temp = q[front];
        front++;
        q[rear++] = temp;
    }
}

void stack_pop() {
    if (front < rear) {
        front++;
    }
}

int stack_top() {
    if (front < rear) {
        return q[front];
    }
    return -1;
}

int main() {
    const char* kw1 = "void Stack::push(int val)";
    const char* kw2 = "q.push(val)";
    const char* kw3 = "void Stack::pop()";
    const char* kw4 = "q.pop()";
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) {
        return 0;
    }

    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        stack_push(val);
    }
    printf("top of element %d\n", stack_top());
    for (int i = 0; i < m; i++) {
        stack_pop();
    }
    printf("top of element %d\n", stack_top());
    return 0;
}
```

Output:

```
5 2
1 2 3 4 5
top of element 5
top of element 3
```

```
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
top of element 5
top of element 11
```